Week 3 作业：算法题、LP 松弛与 GPU

优化算法课程

说明：这周的核心想法是：

- 很多熟悉的算法题都能写成整数规划（ILP）；
- 把整数约束放松后，可以得到线性规划（LP）；
- LP 松弛后可能不再精确，但形式更规整，往往更适合 GPU 并行。

请尽量用自己的话解释：**原问题是什么、LP 松弛后变成了什么、误差从哪里来、为什么更适合 GPU**。

题 1 [基础]　一元线性回归中的梯度下降步长

背景： 考虑一元线性回归模型

$$\hat{y}_i = wx_i,$$

其中参数 $w \in \mathbb{R}$，训练数据为 (x_i, y_i)，$i = 1, \ldots, n$。平方损失函数定义为

$$f(w) = \frac{1}{2n} \sum_{i=1}^{n} (wx_i - y_i)^2.$$

我们用梯度下降更新参数：

$$w_{t+1} = w_t - \eta f'(w_t).$$

(a) 计算目标函数 $f(w)$ 的一阶导数 $f'(w)$ 和二阶导数 $f''(w)$。

(b) 设

$$A = \frac{1}{n} \sum_{i=1}^{n} x_i^2.$$

证明梯度下降更新可以写成

$$w_{t+1} - w^* = (1 - \eta A)(w_t - w^*),$$

其中 w^* 是最优解。

(c) 由上式说明：为了使梯度下降收敛，步长 η 至少应满足什么条件？请给出 η 的最大允许范围。

(d) 用自己的话解释：为什么当数据的 x_i 整体更大时，允许的最大步长会更小？这说明了步长与数据尺度之间有什么关系？

(e)（思考）若把所有输入同时放大为 $x_i' = cx_i$，其中 $c > 0$，那么允许的最大步长会如何变化？

标准答案：

(a) 求导 展开目标函数：

$$f(w) = \frac{1}{2n}\sum_{i=1}^{n}(w^2x_i^2 - 2wx_iy_i + y_i^2).$$

因此

$$f'(w) = \frac{1}{n}\sum_{i=1}^{n}x_i(wx_i - y_i) = \left(\frac{1}{n}\sum_{i=1}^{n}x_i^2\right)w - \frac{1}{n}\sum_{i=1}^{n}x_iy_i.$$

记

$$A = \frac{1}{n}\sum_{i=1}^{n}x_i^2, \qquad B = \frac{1}{n}\sum_{i=1}^{n}x_iy_i,$$

则

$$f'(w) = Aw - B.$$

再求导得

$$f''(w) = A = \frac{1}{n}\sum_{i=1}^{n}x_i^2 \geq 0.$$

若并非所有 x_i 都为 0，则 $A > 0$，目标函数严格凸。

(b) 误差递推式 最优解 w^* 满足一阶最优性条件

$$f'(w^*) = Aw^* - B = 0,$$

故

$$w^* = \frac{B}{A} \qquad (A > 0).$$

梯度下降更新为

$$w_{t+1} = w_t - \eta(Aw_t - B).$$

利用 $B = Aw^*$，可得

$$w_{t+1} = w_t - \eta A(w_t - w^*).$$

两边同时减去 w^*：

$$w_{t+1} - w^* = (1 - \eta A)(w_t - w^*).$$

这就是所求递推关系。

(c) 收敛条件与最大步长范围 由上式可见，设误差 $C = w_t - w^*$，那么上式变成 $C_{t+1} = (1-\eta A)C_t$，误差每一步都被乘上同一个系数 $1-\eta A$。要使 $w_t \to w^*$，必须满足

$$|1-\eta A| < 1.$$

解得

$$-1 < 1-\eta A < 1 \quad \Longleftrightarrow \quad 0 < \eta A < 2.$$

因此当 $A > 0$ 时，梯度下降收敛的充分必要条件是

$$\boxed{0 < \eta < \frac{2}{A}}.$$

所以步长的最大允许范围是开区间 $(0, 2/A)$；临界值 $\eta = 2/A$ 时会来回振荡而不收敛。

(d) 数据尺度与步长的关系 当 x_i 整体更大时，x_i^2 也更大，从而

$$A = \frac{1}{n}\sum_i x_i^2$$

会变大。由于最大允许步长是 $2/A$，所以 A 越大，允许的步长上界越小。

直观上说：数据尺度越大，损失函数沿着参数方向的曲率越大，下降时更容易“迈步过大而越过谷底”，因此必须把步长调小。步长上界与数据的平方尺度成反比。

(e) 输入同时缩放 $x_i' = cx_i$ 时的变化 此时

$$A' = \frac{1}{n}\sum_i (x_i')^2 = \frac{1}{n}\sum_i c^2 x_i^2 = c^2 A.$$

故新的最大允许步长为

$$\eta'_{\max} = \frac{2}{A'} = \frac{2}{c^2 A} = \frac{1}{c^2}\eta_{\max}.$$

因此若把输入整体放大 c 倍，允许的最大步长会缩小为原来的 $1/c^2$ 倍。

题 2 [基础] 最大权二分匹配

背景： 有 n 个用户、m 个广告位。每个广告位最多分给一个用户，每个用户最多得到一个广告位。兼容度矩阵为 $C \in \mathbb{R}^{n\times m}$。平台希望最大化总兼容度。

(a) 用变量 x_{ij} 表示“用户 i 是否分配给广告位 j”，写出该问题的 ILP。

(b) 去掉整数约束，写出 LP 松弛。

(c) 用一句话说明：为什么这个问题的 LP 松弛比较“幸运”，通常不会带来误差?

标准答案：

(a) ILP 形式 令

$$x_{ij} = \begin{cases} 1, & \text{若用户}i\text{ 被分配给广告位}j, \\ 0, & \text{否则。} \end{cases}$$

则最大权二分匹配可以写成：

$$\max \sum_{i=1}^{n} \sum_{j=1}^{m} C_{ij} x_{ij}$$

满足

$$\sum_{j=1}^{m} x_{ij} \le 1, \qquad i = 1, \ldots, n,$$

$$\sum_{i=1}^{n} x_{ij} \le 1, \qquad j = 1, \ldots, m,$$

$$x_{ij} \in \{0, 1\}, \qquad \forall i, j.$$

前两组约束分别表示“每个用户最多匹配一个广告位”和“每个广告位最多给一个用户”。

(b) LP 松弛 去掉整数约束后，得到

$$\max \sum_{i=1}^{n} \sum_{j=1}^{m} C_{ij} x_{ij}$$

满足

$$\sum_{j=1}^{m} x_{ij} \le 1, \qquad i = 1, \ldots, n,$$

$$\sum_{i=1}^{n} x_{ij} \le 1, \qquad j = 1, \ldots, m,$$

$$0 \le x_{ij} \le 1, \qquad \forall i, j.$$

(c) 为什么这个 LP 松弛很“幸运” 一句话说明：**因为二分匹配的约束矩阵是全单模的，LP 的极点解天然就是整数解，所以松弛后通常不会带来误差。**

题 3 [基础] 0-1 背包

背景： 有 n 个物品，第 i 个物品价值为 v_i，重量为 w_i，背包容量为 W。每个物品只能选或不选。

(a) 写出 0-1 背包的 ILP。

(b) 写出对应的 LP 松弛。

(c) 解释为什么 LP 松弛后可能出现“拿半个物品”的现象，因此 LP 最优值一般会比整数最优值更乐观。

标准答案：

(a) 0-1 背包的 ILP 设 $x_i \in \{0,1\}$ 表示是否选择第 i 个物品，则问题可写为

$$\max \sum_{i=1}^{n} v_i x_i$$

满足

$$\sum_{i=1}^{n} w_i x_i \le W, \qquad x_i \in \{0,1\}, \quad i = 1, \ldots, n.$$

(b) LP 松弛 将整数约束放松为区间约束，得到

$$\max \sum_{i=1}^{n} v_i x_i$$

满足

$$\sum_{i=1}^{n} w_i x_i \le W, \qquad 0 \le x_i \le 1, \quad i = 1, \ldots, n.$$

(c) 为什么会出现“拿半个物品” 因为在 LP 松弛里，x_i 不再被限制为只能取 0 或 1，而是允许取任何实数 $0 \le x_i \le 1$。于是模型可能让某个物品只取一部分，例如 $x_i = 0.4$，这在数学上表示“拿了 40% 个物品”。

这会使 LP 的可行域比原来的整数可行域更大，因此 LP 的最优值通常不会更差；对最大化问题来说，它往往会比真正的整数最优值更乐观，所以常被当作一个**上界**。

题 4 [基础] Vertex Cover

背景： 给定无向图 $G = (V, E)$。希望选出尽可能少的顶点，使得每条边至少有一个端点被选中。

(a) 写出 Vertex Cover 的 ILP。

(b) 写出对应的 LP 松弛。

(c) 若 LP 解中有些顶点取值是 1/2，这说明了什么？请用一句话解释“松弛误差”的含义。

标准答案：

(a) Vertex Cover 的 ILP 设对每个顶点 $v \in V$，变量

$$x_v = \begin{cases} 1, & \text{若顶点} v \text{ 被选入覆盖}, \\ 0, & \text{否则。} \end{cases}$$

则 Vertex Cover 可写为

$$\min \sum_{v \in V} x_v$$

满足对每条边 $(u, v) \in E$ 都有

$$x_u + x_v \geq 1,$$

以及

$$x_v \in \{0, 1\}, \qquad \forall v \in V.$$

约束 $x_u + x_v \geq 1$ 表示：每条边至少有一个端点被选中。

(b) LP 松弛 把 $x_v \in \{0, 1\}$ 放松成区间约束，得到

$$\min \sum_{v \in V} x_v$$

满足

$$x_u + x_v \geq 1, \qquad \forall (u, v) \in E,$$
$$0 \leq x_v \leq 1, \qquad \forall v \in V.$$

(c) 当 LP 解里出现 1/2 时说明什么 若某些顶点取值为 1/2，说明 LP 只是在“分数意义”上满足覆盖约束：它允许两个端点各拿出一半来一起覆盖一条边，但在真正的整数问题里，顶点只能选或不选，不能选半个。

因此，所谓“松弛误差”就是：**LP 松弛得到的最优值和原整数问题最优值之间的差距，它来自把离散选择放宽成了连续分数选择。**

题 5 [理解] 背包问题中的松弛误差

背景： 考虑如下 0-1 背包问题：共有 3 个物品，

$$(v_1, w_1) = (14, 6), \qquad (v_2, w_2) = (12, 5), \qquad (v_3, w_3) = (8, 4),$$

背包容量为 $W = 10$。设决策变量 $x_i \in \{0, 1\}$ 表示是否选择物品 i。

(a) 写出该问题的 ILP。

(b) 忽略整数约束，写出对应的 LP 松弛，并用“允许最后一个物品取分数”的方式，计算根节点的 LP 上界。

(c) 以变量 x_1 为第一个分支变量，分别考虑两个子问题：

i. $x_1 = 1$；

ii. $x_1 = 0$。

对每个子问题，计算其 LP 松弛上界，并判断该分支是否可能产生更优整数解。

(d) 根据上面的结果，继续做一步分支定界，找出该背包问题的最优整数解。

(e) 用自己的话解释：分支定界法中的“分支”和“定界”分别是什么意思？为什么 LP 松弛可以用来帮助剪枝？

标准答案：

(a) 写出 ILP 该 0-1 背包问题为

$$\max\ 14x_1 + 12x_2 + 8x_3$$

满足

$$6x_1 + 5x_2 + 4x_3 \le 10, \qquad x_1, x_2, x_3 \in \{0, 1\}.$$

(b) LP 松弛与根节点上界 去掉整数约束后得到

$$\max\ 14x_1 + 12x_2 + 8x_3$$

满足

$$6x_1 + 5x_2 + 4x_3 \le 10, \qquad 0 \le x_1, x_2, x_3 \le 1.$$

按单位重量价值排序：

$$\frac{v_2}{w_2} = \frac{12}{5} = 2.4, \qquad \frac{v_1}{w_1} = \frac{14}{6} \approx 2.33, \qquad \frac{v_3}{w_3} = \frac{8}{4} = 2.$$

因此根节点的 LP 最优解取法是：先取满物品 2，再取物品 1 的一部分。

取 $x_2 = 1$ 后还剩容量 5，于是

$$x_1 = \frac{5}{6}, \qquad x_3 = 0.$$

对应 LP 上界为

$$12 + 14 \cdot \frac{5}{6} = 12 + \frac{35}{3} = \frac{71}{3} \approx 23.67.$$

所以根节点 LP 上界是

$$\boxed{\frac{71}{3} \approx 23.67.}$$

(c) 对两个分支分别计算上界　分支 i： $x_1 = 1$

此时剩余容量为 $10 - 6 = 4$。子问题 LP 为

$$\max\ 14 + 12x_2 + 8x_3$$

满足

$$5x_2 + 4x_3 \le 4, \qquad 0 \le x_2, x_3 \le 1.$$

由于物品 2 的单位重量价值更高，LP 会优先分数取物品 2：

$$x_2 = \frac{4}{5}, \qquad x_3 = 0.$$

故上界为

$$14 + 12 \cdot \frac{4}{5} = 14 + 9.6 = 23.6.$$

整数可行解中，此分支最好是 $x_1 = 1, x_3 = 1, x_2 = 0$，价值为

$$14 + 8 = 22.$$

因此该分支**仍有可能**产生较优整数解，需要继续往下分。

分支 ii： $x_1 = 0$

此时子问题为

$$\max\ 12x_2 + 8x_3$$

满足

$$5x_2 + 4x_3 \le 10, \qquad 0 \le x_2, x_3 \le 1.$$

因为物品 2 和物品 3 可以同时装下（总重量 $5 + 4 = 9 \le 10$），所以该分支 LP 上界就是

$$12 + 8 = 20.$$

而且这已经是整数解，对应 $x_2 = 1, x_3 = 1$。所以此分支的最优整数值也是 20。

若当前已有可行整数解价值 22，则该分支上界 $20 < 22$，可以直接剪枝。

(d) 继续一步分支定界，找最优整数解　在 $x_1 = 1$ 分支下继续分：

- 若 $x_1 = 1, x_2 = 1$，则总重量 $6 + 5 = 11 > 10$，不可行，直接剪枝；
- 若 $x_1 = 1, x_2 = 0$，则还可取 $x_3 = 1$，得到整数解

  $$(x_1, x_2, x_3) = (1, 0, 1),$$

  目标值为

  $$14 + 8 = 22.$$

  再结合 $x_1 = 0$ 分支只能达到 20，可知全局最优整数解为

$$\boxed{(x_1, x_2, x_3) = (1, 0, 1), \qquad z^* = 22.}$$

(e) 分支与定界的含义　**分支**是指：对某个还没有确定整数取值的变量，强制它走不同情形（如 $x_1 = 0$ 或 $x_1 = 1$），从而把原问题拆成若干更小的子问题。

定界是指：对每个子问题先求一个容易得到的上界（这里用 LP 松弛），如果这个上界都不可能超过当前最好整数解，那么这个子问题就没必要再往下搜，可以剪枝。

LP 松弛之所以能帮助剪枝，是因为它比整数问题容易求，而且对最大化问题给出的是一个可靠上界。

题 6 [理解]　Vertex Cover 的简单舍入

背景：对 Vertex Cover 的 LP 松弛，考虑如下简单规则：若 $x_v^* \geq 1/2$，就把顶点 v 选入解中。

(a) 说明为什么这个规则一定得到一个合法的顶点覆盖。

(b) 证明该规则得到的解大小不超过 $2\sum_v x_v^*$。

(c) 用自己的话解释：为什么这说明 LP 松弛虽然不精确，但仍然能给出一个“还不错”的近似解？

标准答案：

(a) 为什么一定得到合法的顶点覆盖　对任意边 $(u, v) \in E$，LP 松弛约束给出

$$x_u^* + x_v^* \geq 1.$$

因此不可能同时有 $x_u^* < 1/2$ 且 $x_v^* < 1/2$，否则两者之和会小于 1，与约束矛盾。

所以，对每一条边，至少有一个端点满足 $x^* \geq 1/2$。按规则把所有满足 $x_v^* \geq 1/2$ 的点都选进去后，每条边至少有一个端点被选中，于是得到的一定是一个合法的顶点覆盖。

(b) 证明解大小不超过 $2\sum_v x_v^*$

$$\text{设} S = \{v \in V : \ x_v^* \geq 1/2\}.$$

我们要证明：

$$|S| \leq 2\sum_{v \in V} x_v^*.$$

首先，由集合大小的定义可知，

$$|S| = \sum_{v \in S} 1.$$

对于任意 $v \in S$，由于 $x_v^* \geq 1/2$，所以

$$2x_v^* \geq 1,$$

即

$$1 \le 2x_v^*.$$

对所有 $v \in S$ 将上述不等式求和，得到

$$\sum_{v \in S} 1 \le \sum_{v \in S} 2x_v^*.$$

因此，

$$|S| \le 2 \sum_{v \in S} x_v^*.$$

又因为 $S \subseteq V$，所以

$$\sum_{v \in S} x_v^* \le \sum_{v \in V} x_v^*.$$

于是可得

$$|S| \le 2 \sum_{v \in V} x_v^*.$$

(c) 为什么这说明 LP 松弛能给出“还不错”的近似解 因为 $\sum_v x_v^*$ 是 LP 松弛的最优值，它一定不超过真正整数最优值 OPT，所以

$$|S| \le 2 \sum_{v} x_v^* \le 2\,\mathrm{OPT}.$$

这说明用 LP 解做简单舍入后，虽然不一定得到最优整数解，但得到的解最多只比最优解大两倍，因此它提供了一个有理论保证的 2-近似。换句话说，LP 松弛虽然不精确，却给出了一个很好的“下界 + 舍入模板”。

题 7 [理解] 最短路为什么几乎没有松弛误差

背景： 在有向图中，最短路可以写成

$$\max d_t \qquad \text{s.t.} \quad d_v - d_u \le w_{(u,v)}, \quad d_s = 0.$$

(a) 这里的变量 d_v 表示什么？

(b) 为什么这个模型本身已经是连续的，不像背包或 Vertex Cover 那样需要“选或不选”？

(c) 用一句话比较：为什么有些问题松弛后误差大，而最短路这种问题几乎没有这个问题？

标准答案：

(a) 变量 d_v 表示什么 变量 d_v 可以理解为从源点 s 到顶点 v 的“距离势函数”或“最短路长度估计”。

约束

$$d_v - d_u \le w_{(u,v)}$$

等价于

$$d_v \le d_u + w_{(u,v)},$$

表示：沿任意一条边走过去，终点距离不能超过“起点距离 + 边权”。因此任意可行解都给出了对最短路长度的一组一致上界，而目标是把 d_t 尽量推大，最终正好推到从 s 到 t 的最短路长度。

(b) 为什么它本身已经是连续模型 这个模型的变量只是各个顶点的实数距离 d_v，没有出现“某条边选或不选”“某个点进或不进”这类二元决策。因此它从一开始就是连续优化问题，不需要像背包或 Vertex Cover 那样再额外引入整数变量。

(c) 为什么最短路几乎没有松弛误差 一句话比较：**背包和 Vertex Cover 的难点来自离散组合选择，而最短路的核心结构本来就是连续的差分约束，所以不存在把整数变量放松成分数变量后带来的那种松弛误差。**

题 8 [讨论] 从离散算法到规整迭代

背景：很多 CPU 上的经典算法（如 Dijkstra、动态规划、分支搜索）都依赖复杂控制流；而 GPU 更擅长做大量结构相同的并行计算。

(a) 请比较下面两类计算的区别：

i. 用堆实现 Dijkstra；

ii. 对所有边同时做一次 Bellman-Ford 松弛。

哪一种更适合 GPU？为什么？

(b) 请比较下面两类计算的区别：

i. 用回溯法精确求解 0-1 背包：不断尝试“选 / 不选”每个物品；

ii. 将 0-1 背包放松成连续变量问题后，同时计算所有物品变量的更新方向。

哪一种更适合 GPU？为什么？

标准答案：

(a) **图计算：Dijkstra 顺序堆维护 vs. Bellman-Ford 全边锁步松弛**：

- **Dijkstra**：1. **步骤分拆**： 初始化距离； 将源点入堆； **循环**：从堆中弹出权值最小的点 u，遍历其邻居 v，若 $d(u) + w < d(v)$ 则更新并入堆。2. **GPU 并行化困境**：虽然上述循环看起来有很多节点，但核心操作处于堆的维护中。在 GPU 的 SIMT（单指令多线程）架构下，指令发射器发出一行 "Pop min" 指令，由于全局只有一个最小点，这就导致 32 个线程中只有 1 个在工作，其他 31 个都在等待（严重的分支空转）。此外，堆操作涉及频繁的内存随机交换，无法形成合并访问 (Coalesced Access)，显存带宽利用率极低。
- **Bellman-Ford**：1. **步骤分拆**：进行 $V - 1$ 次大循环，每一轮对边集 E 中的所有边 (u, v) **同时执行**：$d(v) = \min(d(v), d(u) + w_{uv})$。2. **GPU 物理映射**：我们将每一条边映射给一个固定的 GPU 线程。在算力的海洋里，GPU 不需要考虑"谁先谁后"，它每一轮都发出统一的"乘加 + 最小值"指令。所有线程在锁步（Lock-step）执行下，一次时钟周期内就完成了成千上万条边的松弛。3. **结论**：Dijkstra 的 $O(E \log V)$ 是节约了计算量，但破坏了规整性；Bellman-Ford 的 $O(VE)$ 增加了冗余计算，但其运算结构完美契合 GPU 的吞吐量模型。

(b) **决策优化：回溯搜索树 vs. 连续空间张量迭代**：

- **回溯法/DFS**：1. **步骤分拆**： 选择物品； 递归分支"选"； 递归分支"不选"； 满足容量则保留，否则剪枝回溯。2. **GPU 并行化困境**：当我们尝试让不同线程并行搜索不同分支时，会遭遇最致命的 Warp Divergence（线程束分歧）。例如，线程 A 正在处理一个极其深且复杂的子树，而线程 B 因为剪枝条件触发已经完成了搜索。在 GPU 物理层，线程 B 无法去跑新任务，它必须在原地"陪跑"线程 A 直到整组指令结束。这种"一人慢则全队慢"的特性让搜索类算法在 GPU 上效率极低。
- **连续松弛更新**：1. **步骤分拆**： 将离散选择 $x_i \in \{0, 1\}$ 松弛为密度分布 $x_i \in [0, 1]$； 构建一个连续可微的损失函数（势能面）； 使用同步梯度下降更新向量：$\mathbf{X}_{new} = \mathbf{X} - \eta \nabla L$。2. **GPU 物理映射**：这完全变成了矩阵乘法与张量加法。所有决策变量分布在显存的连续地址上。GPU 的 SM 单元可以一次性搬运大块数据进入 L1 缓存，每个物品的更新逻辑完全一致。没有分支、没有回溯、没有等待，只有整齐划一的浮点运算，让 GPU 算力得到 100% 的释放。3. **结论**：连续松弛将"逻辑冲突"转化为了"数值演化"。即便需要更多次的迭代才能收敛，由于 GPU 单次迭代的极速优势，其总耗时通常远低于传统的搜索算法。

题 9 [讨论] 编辑距离与反对角线并行

背景： 编辑距离的经典动态规划为

$$\mathrm{dp}[i][j] = \min\Big(\mathrm{dp}[i-1][j]+1,\ \mathrm{dp}[i][j-1]+1,\ \mathrm{dp}[i-1][j-1]+\mathbf{1}[s_i \neq t_j]\Big).$$

(a) 解释为什么同一条反对角线上的状态可以同时更新。

(b) 说明这类“分层更新”的结构为什么适合 GPU。

(c) 用一句话说明：这道题虽然本质上是 DP，但为什么也能看成“更规整的并行松弛过程”？

标准答案：

(a) 为什么同一条反对角线上的状态可以同时更新 对状态 (i,j)，其转移只依赖于

$$(i-1,j), \qquad (i,j-1), \qquad (i-1,j-1).$$

若固定同一条反对角线 $i+j=k$，则这三个依赖状态分别位于

$$(i-1)+j=k-1, \qquad i+(j-1)=k-1, \qquad (i-1)+(j-1)=k-2.$$

也就是说，它们都来自更早的反对角线，而不来自当前这条反对角线本身。因此，**同一条反对角线上的状态彼此之间没有依赖关系，只依赖前面反对角线的结果**。所以当我们固定 $i+j=k$ 时，这一层上的所有状态都可以同时更新。

(b) 为什么这种“分层更新”适合 GPU 因为每一层里的状态更新公式完全相同，都是做三者取最小值：

$$\min\{\mathrm{dp}[i-1][j]+1,\ \mathrm{dp}[i][j-1]+1,\ \mathrm{dp}[i-1][j-1]+\mathbf{1}[s_i \neq t_j]\}.$$

这意味着：

- 同一层上每个线程执行相同的算子；
- 依赖关系整齐，只需要层与层之间同步；
- 内存访问比回溯搜索更规则；
- 不需要复杂的分支控制。

所以它很适合 GPU 的大规模同步并行。

(c) 为什么也能看成“更规整的并行松弛过程” 一句话说明：**虽然编辑距离来源于动态规划，但每次更新本质上都是利用已知的邻居状态去“松弛”当前状态的最优值，因此它也可以看成一种按反对角线分层推进的规整并行松弛。**

直观地说，原本 DP 是按顺序填表；改成按反对角线分层后，就变成了“分轮次推进、同一轮批量更新、只依赖前几轮结果”的规整更新过程，这和很多并行松弛算法的结构是相似的。

题 10 [编程] 值迭代：CPU 与 GPU 对比

背景： 值迭代的核心更新为

$$V(s) \leftarrow \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right].$$

当对所有状态同时更新时，这一过程很适合向量化实现。

任务：在一个网格 MDP 上，实现 CPU 版和 GPU 版值迭代，并比较运行时间。

(a) 实现一个网格 MDP 生成函数，返回奖励矩阵 R 和转移张量 P。

(b) 用 NumPy 实现 CPU 版值迭代。

(c) 用 PyTorch 实现 GPU 版值迭代，并正确计时。

(d) 在不同网格规模下比较 CPU 与 GPU 的运行时间，并回答：

i. 状态数变大后，GPU 何时开始占优？

ii. 每轮迭代中，最耗时的部分通常是什么？

iii. 为什么值迭代比很多“带大量分支的搜索算法”更适合 GPU？

参考代码框架如下：

```python
import numpy as np
import torch
import time

def make_grid_mdp(grid_size, gamma=0.99, seed=42):
    """
    生成 grid_size x grid_size 的网格 MDP。
    返回：
        R: shape (S, A)
        P: shape (A, S, S)
    """
```

```python
    pass

def value_iteration_cpu(R, P, gamma=0.99, tol=1e-6, max_iter=10000):
    S, A = R.shape
    V = np.zeros(S)
    start = time.time()
    for it in range(max_iter):
        Q = R + gamma * np.einsum('aij,j->ia', P, V)
        V_new = Q.max(axis=1)
        if np.max(np.abs(V_new - V)) < tol:
            return V_new, it + 1, time.time() - start
        V = V_new
    return V, max_iter, time.time() - start

def value_iteration_gpu(R_np, P_np, gamma=0.99, tol=1e-6,
                        max_iter=10000, device='cuda'):
    R = torch.tensor(R_np, dtype=torch.float32, device=device)
    P = torch.tensor(P_np, dtype=torch.float32, device=device)
    V = torch.zeros(R_np.shape[0], dtype=torch.float32, device=device)

    torch.cuda.synchronize()
    start = time.time()

    for it in range(max_iter):
        Q = R + gamma * torch.einsum('aij,j->ia', P, V)
        V_new = Q.max(dim=1).values
        if torch.max(torch.abs(V_new - V)).item() < tol:
            torch.cuda.synchronize()
            return V_new.cpu().numpy(), it + 1, time.time() - start
        V = V_new

    torch.cuda.synchronize()
    return V.cpu().numpy(), max_iter, time.time() - start
```

标准答案:

实验背景与环境准备 本实验演示如何在网格 MDP 上实现值迭代（Value Iteration），并对比 CPU（NumPy）与 GPU（PyTorch）的运行效率。

首先导入必要的库并检测可用的硬件设备：

```python
import numpy as np
import torch
import time
import matplotlib.pyplot as plt

# 检测设备
device = "mps" if torch.backends.mps.is_available() else ("cuda" if torch.cuda.is_available() else "cpu")
print(f"当前使用的 GPU 设备: {device}")
```

实现 Grid MDP 生成函数 生成一个 $N \times N$ 的网格。状态总数为 $S = N^2$。动作空间为 4（上、下、左、右）。

- 转移张量 P：形状为 (A, S, S)，其中 $P[a, s, s']$ 表示在状态 s 采取动作 a 转移到 s' 的概率。
- 奖励矩阵 R：形状为 (S, A)，表示在状态 s 采取动作 a 获得的奖励。

设定右下角为目标状态，进入该状态奖励为 1.0。

```python
def make_grid_mdp(grid_size, gamma=0.99, seed=42):
    """
    生成 grid_size x grid_size 的网格 MDP。
    返回:
        R: shape (S, A) - 奖励矩阵
        P: shape (A, S, S) - 转移张量
    """
    S = grid_size * grid_size
    A = 4  # 0: Up, 1: Down, 2: Left, 3: Right

    P = np.zeros((A, S, S))
    R = np.zeros((S, A))

    for r in range(grid_size):
        for c in range(grid_size):
            s = r * grid_size + c
            for a in range(A):
```

```
                next_r, next_c = r, c
                if a == 0: next_r = max(0, r - 1)
                elif a == 1: next_r = min(grid_size - 1, r + 1)
                elif a == 2: next_c = max(0, c - 1)
                elif a == 3: next_c = min(grid_size - 1, c + 1)

                next_s = next_r * grid_size + next_c
                P[a, s, next_s] = 1.0

                if next_r == grid_size - 1 and next_c == grid_size - 1 and (r != grid_size - 1 or c != grid_size - 1):
                    R[s, a] = 1.0

    return R, P
```

实现值迭代算法 值迭代的核心更新公式为:

$$V(s) \leftarrow \max_a \left[r(s,a) + \gamma \sum_{s'} P(s'|s,a) V(s') \right]$$

利用向量化操作（einsum），可以极大地加速这一过程。

```
def value_iteration_cpu(R, P, gamma=0.99, tol=1e-6, max_iter=10000):
    """
    使用 NumPy 在 CPU 上进行值迭代。
    """
    S, A = R.shape
    V = np.zeros(S)
    start = time.time()

    for it in range(max_iter):
        Q = R + gamma * np.einsum('aij,j->ia', P, V)
        V_new = Q.max(axis=1)

        if np.max(np.abs(V_new - V)) < tol:
            return V_new, it + 1, time.time() - start

        V = V_new

    return V, max_iter, time.time() - start

```

```python
def value_iteration_gpu(R_np, P_np, gamma=0.99, tol=1e-6, max_iter=10000, device='cuda'):
    """
    使用 PyTorch 在 GPU 上进行值迭代。
    """
    R = torch.tensor(R_np, dtype=torch.float32, device=device)
    P = torch.tensor(P_np, dtype=torch.float32, device=device)
    V = torch.zeros(R_np.shape[0], dtype=torch.float32, device=device)

    if "cuda" in str(device):
        torch.cuda.synchronize()

    start = time.time()

    for it in range(max_iter):
        Q = R + gamma * torch.einsum('aij,j->ia', P, V)
        V_new = Q.max(dim=1).values
        if torch.max(torch.abs(V_new - V)).item() < tol:
            if "cuda" in str(device):
                torch.cuda.synchronize()
            return V_new.cpu().numpy(), it + 1, time.time() - start
        V = V_new

    if "cuda" in str(device):
        torch.cuda.synchronize()

    return V.cpu().numpy(), max_iter, time.time() - start
```

性能测试与对比 通过测试不同网格规模（10×10 到 80×80）下的耗时，可以观察到随着状态数增加，GPU 的优势逐渐显现。

```python
sizes = [10, 20, 40, 60, 80]
cpu_times = []
gpu_times = []

for size in sizes:
    R, P = make_grid_mdp(size)
    _, iters, t_cpu = value_iteration_cpu(R, P)
    cpu_times.append(t_cpu)
```

```
    _, _, t_gpu = value_iteration_gpu(R, P, device=device)
    gpu_times.append(t_gpu)

# 结果可视化（略去绘图代码，见交互式环境）
```

问题讨论

状态数变大后，GPU 何时开始占优？ 通常情况下，在状态数较少（如 10×10）时，CPU 可能更快，因为 GPU 的数据传输和算子调度存在额外开销。当状态数达到数千（如网格规模 60×60 以上）时，GPU 的大规模并行计算优势将抵消这些开销，开始显著优于 CPU。

每轮迭代中，最耗时的部分通常是什么？ 最耗时的部分是转移概率张量与价值向量的乘法运算，即公式中的 $\sum_{s'} P(s'|s,a)V(s')$（在代码中表现为 `einsum` 操作）。该操作的计算复杂度为 $O(A \times S^2)$，随状态数平方级增长。

为什么值迭代比“带大量分支的搜索算法”更适合 GPU？ 值迭代是数据并行的，每一轮迭代中所有状态的更新相互独立，且执行的是相同的数学运算，非常契合 GPU 的 SIMD（单指令多数据流）架构。而带分支的搜索算法中，不同路径的计算逻辑不一致，会导致 GPU 执行流的分叉（Thread Divergence），从而降低硬件利用率。